

Week 6 Problem Set

Minimum Spanning Trees, Shortest Paths, Maximum Flows

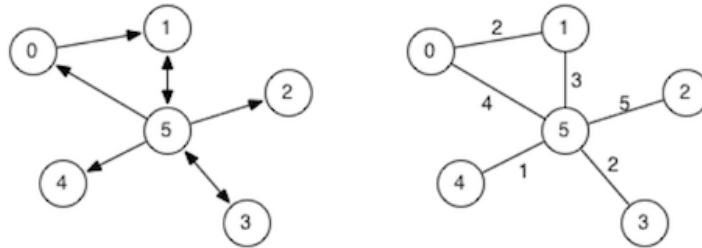
[\[Show with no answers\]](#) [\[Show with all answers\]](#)

Please note:

- The contents of this week's lecture and homework will *not* be covered by the mid-term test.
- Because of the mid-term test on Monday, October 28, the deadline for this week's programs will be on the day of the next lecture: **Wednesday, October 30th.**

1. (Graph representations)

- a. Consider the following graphs, where bi-directional edges are depicted as two-way arrows rather than having two separate edges going in opposite directions:



For each of the graphs show the concrete data structures if the graph was implemented via:

- adjacency matrix representation (assume full $V \times V$ matrix)
- adjacency list representation (if non-directional, include both (v,w) and (w,v))

- b. Consider the following map of streets in the Sydney CBD:

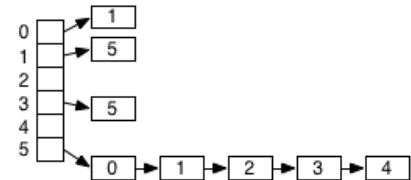


Represent this as a directed graph, where intersections are vertices and the connecting streets are edges. Ensure that the directions on the edges correctly reflect any one-way streets (this is a driving map, not a walking map). You only need to make a graph which includes the intersections marked with red letters. Some things that don't show on the map: Castlereagh St is one-way heading south and Curtin Pl is a little laneway that you can't drive down.

For each of the following pairs of intersections, indicate whether there is a path from the first to the second. Show a simple path if there is one. If there is more than one simple path, show two different paths.

- from intersection "D" on Margaret St to intersection "L" on Pitt St
- from intersection "J" to the corner of Margaret St and York St (intersection "A")
- from intersection "P" on Castlereagh St to the corner of Margaret St and Wynyard Ln ("C")
- from the intersection of Castlereagh St and Hunter St ("M") to intersection "H" at Wynyard Park

[\[hide answer\]](#)



```

graph TD
    A --> B
    A --> H
    B --> C
    B --> I
    C --> D
    C --> J
    D --> F
    E --> G
    F --> G
    F --> K
    G --> L
    G --> M
    M --> N
    M --> P
    I --> H
    I --> J
    J --> I
    K --> F
    L --> G
    P --> M
    N --> M

```

tc	[A]	[B]	[C]	[D]	[E]	[F]	[G]	[H]	[I]	[J]	[K]	[L]	[M]	[N]	[P]
[A]	0	1	0	0	0	0	0	1	0	0	0	0	0	0	0
[B]	1	0	1	0	0	0	0	0	0	0	0	0	0	0	0
[C]	0	1	0	1	0	0	0	0	0	1	0	0	0	0	0
[D]	0	0	1	0	0	1	0	0	0	0	0	0	0	0	0
[E]	0	0	0	0	0	0	1	0	0	0	0	0	0	0	0
[F]	0	0	0	1	0	0	1	0	0	0	1	0	0	0	0
[G]	0	0	0	0	0	1	0	0	0	0	0	1	1	0	0
[H]	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
[I]	0	1	0	0	0	0	0	1	0	0	0	0	0	0	0
[J]	0	0	0	0	0	0	0	0	1	0	0	0	0	0	0
[K]	0	0	0	0	0	1	0	0	0	0	0	0	0	0	0

[L]	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
[M]	0	0	0	0	0	0	1	0	0	0	0	0	0	1	1
[N]	0	0	0	0	0	0	0	0	0	0	0	0	1	0	0
[P]	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

After the first iteration:

tc	[A]	[B]	[C]	[D]	[E]	[F]	[G]	[H]	[I]	[J]	[K]	[L]	[M]	[N]	[P]
[A]	0	1	0	0	0	0	0	1	0	0	0	0	0	0	0
[B]	1	1	1	0	0	0	0	1	0	0	0	0	0	0	0
[C]	0	1	0	1	0	0	0	0	0	1	0	0	0	0	0
[D]	0	0	1	0	0	1	0	0	0	0	0	0	0	0	0
[E]	0	0	0	0	0	0	1	0	0	0	0	0	0	0	0
[F]	0	0	0	1	0	0	1	0	0	0	1	0	0	0	0
[G]	0	0	0	0	0	1	0	0	0	0	0	1	1	0	0
[H]	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
[I]	0	1	0	0	0	0	0	1	0	0	0	0	0	0	0
[J]	0	0	0	0	0	0	0	0	1	0	0	0	0	0	0
[K]	0	0	0	0	0	1	0	0	0	0	0	0	0	0	0
[L]	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
[M]	0	0	0	0	0	0	1	0	0	0	0	0	0	1	1
[N]	0	0	0	0	0	0	0	0	0	0	0	0	1	0	0
[P]	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

This matrix represents the existence of all directed paths:

- of length 1,
- of length >1 that go through vertex A as the only non-endpoint ($B \rightarrow A \rightarrow B$ and $B \rightarrow A \rightarrow H$).

After the sixth iteration:

tc	[A]	[B]	[C]	[D]	[E]	[F]	[G]	[H]	[I]	[J]	[K]	[L]	[M]	[N]	[P]
[A]	1	1	1	1	0	1	1	1	0	1	1	0	0	0	0
[B]	1	1	1	1	0	1	1	1	0	1	1	0	0	0	0
[C]	1	1	1	1	0	1	1	1	0	1	1	0	0	0	0
[D]	1	1	1	1	0	1	1	1	0	1	1	0	0	0	0
[E]	0	0	0	0	0	0	1	0	0	0	0	0	0	0	0
[F]	1	1	1	1	0	1	1	1	0	1	1	0	0	0	0
[G]	1	1	1	1	0	1	1	1	0	1	1	1	1	0	0
[H]	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
[I]	1	1	1	1	0	1	1	1	0	1	1	0	0	0	0
[J]	0	0	0	0	0	0	0	0	1	0	0	0	0	0	0
[K]	1	1	1	1	0	1	1	1	0	1	1	0	0	0	0
[L]	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
[M]	0	0	0	0	0	0	1	0	0	0	0	0	0	1	1
[N]	0	0	0	0	0	0	0	0	0	0	0	0	1	0	0
[P]	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

This matrix represents the existence of all directed paths

- of length 1,
- of length >1 that go through one or more of the vertices A,B,C,D,E,F as non-endpoints.

After the last iteration:

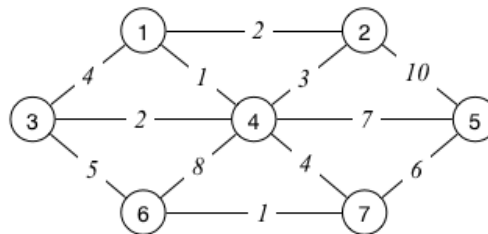
tc	[A]	[B]	[C]	[D]	[E]	[F]	[G]	[H]	[I]	[J]	[K]	[L]	[M]	[N]	[P]
[A]	1	1	1	1	0	1	1	1	1	1	1	1	1	1	1
[B]	1	1	1	1	0	1	1	1	1	1	1	1	1	1	1
[C]	1	1	1	1	0	1	1	1	1	1	1	1	1	1	1

[D]	1	1	1	1	0	1	1	1	1	1	1	1	1	1
[E]	1	1	1	1	0	1	1	1	1	1	1	1	1	1
[F]	1	1	1	1	0	1	1	1	1	1	1	1	1	1
[G]	1	1	1	1	0	1	1	1	1	1	1	1	1	1
[H]	0	0	0	0	0	0	0	0	0	0	0	0	0	0
[I]	1	1	1	1	0	1	1	1	1	1	1	1	1	1
[J]	1	1	1	1	0	1	1	1	1	1	1	1	1	1
[K]	1	1	1	1	0	1	1	1	1	1	1	1	1	1
[L]	0	0	0	0	0	0	0	0	0	0	0	0	0	0
[M]	1	1	1	1	0	1	1	1	1	1	1	1	1	1
[N]	1	1	1	1	0	1	1	1	1	1	1	1	1	1
[P]	0	0	0	0	0	0	0	0	0	0	0	0	0	0

The final matrix represents the existence of all directed paths through any of the vertices as non-endpoints, i.e. the transitive closure.

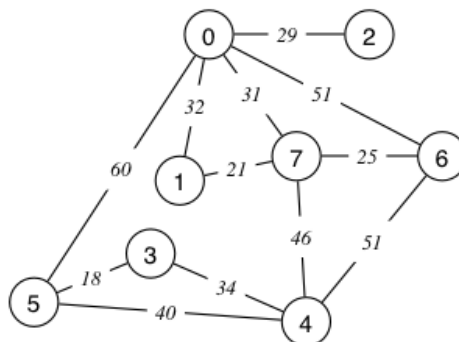
3. (Minimum spanning trees)

a. Show how Kruskal's algorithm would construct the MST for the following graph:



How many edges do you have to consider?

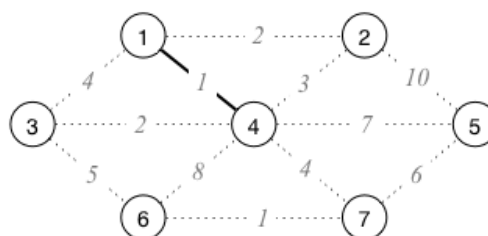
- b. For a graph $G=(V,E)$, what is the least number of edges that might need to be considered by Kruskal's algorithm, and what is the most number of edges? Add one vertex and edge to the above graph to force Kruskal's algorithm to the worst case.
- c. Trace the execution of Prim's algorithm to compute a minimum spanning tree on the following graph:



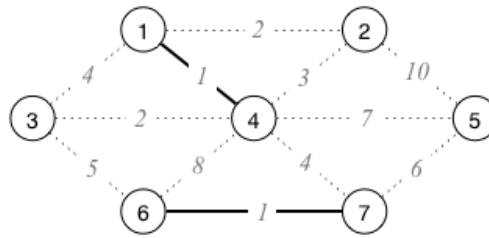
Choose a random vertex to start with. Draw the resulting minimum spanning tree.

[\[hide answer\]](#)

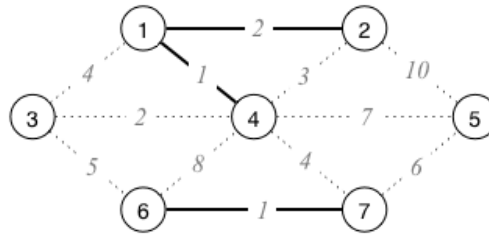
- a. In the first iteration of Kruskal's algorithm, we could choose either 1-4 or 6-7, since both edges have weight 1. Assume we choose 1-4. Since its inclusion produces no cycles, we add it to the MST (non-existent edges are indicated by dotted lines):



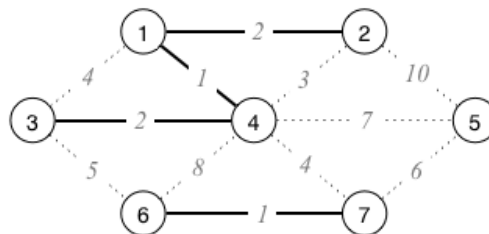
In the next iteration, we choose 6-7. Its inclusion produces no cycles, so we add it to the MST:



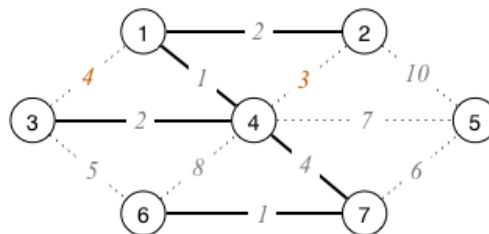
In the next iteration, we could choose either 1-2 or 3-4, since both edges have weight 2. Assume we choose 1-2. Since its inclusion produces no cycles, we add it to the MST:



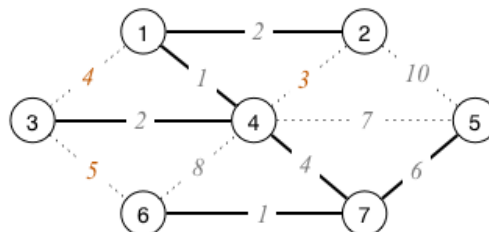
In the next iteration, we choose 3-4. Its inclusion produces no cycles, so we add it to the MST:



In the next iteration, we would first consider the lowest-cost unused edge. This is 2-4, but its inclusion would produce a cycle, so we ignore it. We then consider 1-3 and 4-7 which both have weight 4. If we choose 1-3, that produces a cycle so we ignore that edge. If we add 4-7 to the MST, there is no cycle and so we include it:



Now the lowest-cost unused edge is 3-6, but its inclusion would produce a cycle, so we ignore it. We then consider 5-7. If we add 5-7 to the MST, there is no cycle and so we include it:



At this stage, all vertices are connected and we have a MST.

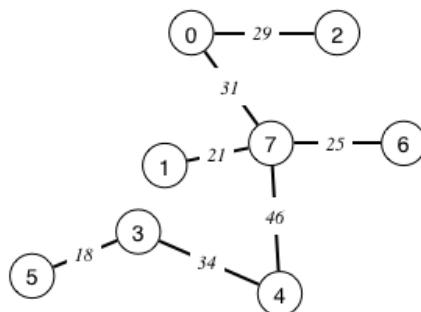
For this graph, we considered 9 of the 12 possible edges in determining the MST.

- For a graph with V vertices and E edges, the best case would be when the first $V-1$ edges we consider are the lowest cost edges and none of these edges leads to a cycle. The worst case would be when we had to consider all E edges. If we added a vertex 8 to the above graph, and connected it to vertex 5 with edge cost 11 (or any cost larger than all the other edge costs in the graph), we would need to consider all edges to construct the MST.
- If we start at node 5, for example, edges would be found in the following order:

- 5-3
- 3-4
- 4-7
- 1-7

- 6-7
- 0-7
- 0-2

The minimum spanning tree:



4. (Priority queue for Dijkstra's algorithm)

Assume that a priority queue for Dijkstra's algorithm is represented by a global variable `PQueue` of the following structure:

```

#define MAX_NODES 1000
typedef struct {
    int item[MAX_NODES]; // array of vertices currently in queue
    int length;           // #values currently stored in item[] array
} PQueueT;

static PQueueT PQueue; // defines the Priority Queue Object
  
```

Further assume that vertices are stored in the `item[]` array in the priority queue in *no* particular order. Rather, the `item[]` array acts like a set, and the priority is determined by reference to a `priority[0..nV-1]` array.

If the priority queue `PQueue` is initialised by:

```

// set up empty priority queue
void PQueueInit() {
    PQueue.length = 0;
}
  
```

and new elements are added as follows:

```

// insert vertex v into priority queue
// no effect if v is already in the queue
void joinPQueue(Vertex v) {
    assert(PQueue.length < MAX_NODES); // ensure queue ADO is not full
    int i = 0;
    while (i < PQueue.length && PQueue.item[i] != v) // check if v already in queue
        i++;
    if (i == PQueue.length) { // v not found => add it at the end
        PQueue.item[PQueue.length] = v;
        PQueue.length++;
    }
}
  
```

then give the implementation of the following functions in C for a priority queue ADO:

```

// remove the highest priority vertex from PQueue
// remember: highest priority = lowest value priority[v]
// returns the removed vertex
Vertex leavePQueue(int priority[]) {
    ...
}

// check if priority queue PQueue is empty
bool PQueueIsEmpty() {
    ...
}
  
```

Download the ADO header `PQueue.h` and the incomplete implementation file `PQueue.c` to get you started, along with the Weighted Graph ADT (`WGraph.h`, `WGraph.c`) from the lecture. Do not change the header files or the weighted graph ADT.

Here is a simple [client program](#) to demonstrate the functionality of the priority queue:

```

// PQueue client ... COMP9024 19T3
#include <stdio.h>
#include "PQueue.h"

int main(void) {
    int distance[2] = { 90, 24 };
  
```

```

PQueueInit();
joinPQueue(0); // vertex 0 (distance = 90) joins the queue
joinPQueue(1); // vertex 1 (distance = 24) joins the queue

while (!PQueueIsEmpty()) {
    printf("Dequeuing %d.\n", leavePQueue(distance));
}
return 0;
}

```

The expected output is:

```

prompt$ ./PQueueClient
Dequeuing 1.
Dequeuing 0.

```

We have created a script that can automatically test your program. To run this test you can execute the dryrun program that corresponds to this exercise. It expects to find a file named `PQueue.c` in the current directory that provides an implementation of a priority queue ADO with the four functions shown above (`PQueueInit`, `JoinPQueue`, `LeavePQueue`, `PQueueIsEmpty`).

You can use dryrun as follows:

```

prompt$ 9024 dryrun PQueue

```

[hide answer]

The following are possible implementations for leaving the priority queue and checking whether it's empty:

```

#define VERY_HIGH_VALUE 999999

int leavePQueue(int priority[]) {
    assert(PQueue.length > 0);

    int i, bestIndex = 0, bestVertex = PQueue.item[0], bestWeight = VERY_HIGH_VALUE;
    for (i = 0; i < PQueue.length; i++) { // find i with min priority[item[i]]
        if (priority[PQueue.item[i]] < bestWeight) {
            bestIndex = i;
            bestWeight = priority[PQueue.item[i]];
            bestVertex = PQueue.item[i]; // vertex with lowest value so far
        }
    }
    PQueue.length--;
    PQueue.item[bestIndex] = PQueue.item[PQueue.length]; // replace dequeued node
                                                         // by last element in array

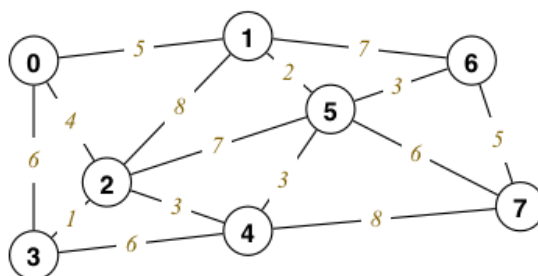
    return bestVertex;
}

bool PQueueIsEmpty() {
    return (PQueue.length == 0);
}

```

5. (Dijkstra's algorithm)

a. Consider the following graph:



What is the shortest path from vertex 0 to vertex 3? From vertex 0 to vertex 7? How did you determine these?

b. Trace the execution of Dijkstra's algorithm on the graph from the previous question to compute the minimum distances from source node 0 to all other vertices.

Show the values of `vSet`, `dist[ ]` and `pred[ ]` after each iteration.

c. Download and complete the incomplete implementation [dijkstra.c](#) of Dijkstra's algorithm that uses your priority queue for Exercise 4 and the Weighted Graph ADT ([WGraph.h](#), [WGraph.c](#)) from the lecture. The program:

- prompts the user for the number of nodes in a graph,
- prompts the user for a source node,
- builds a weighted *undirected* graph from user input (adds every input edge in both directions),
- uses the priority ADO to compute and output the distance and a shortest path from the source to every vertex of the graph (**needs to be implemented**).

An example of the program executing is shown below. The input graph consists of a simple triangle along with a fourth, disconnected node.

```
prompt$ ./dijkstra
Enter the number of vertices: 4
Enter the source node: 0
Enter an edge (from): 0
Enter an edge (to): 1
Enter the weight: 42
Enter an edge (from): 0
Enter an edge (to): 2
Enter the weight: 25
Enter an edge (from): 1
Enter an edge (to): 2
Enter the weight: 14
Enter an edge (from): done
Done.
0: distance = 0, shortest path: 0
1: distance = 39, shortest path: 0-2-1
2: distance = 25, shortest path: 0-2
3: no path
```

Note that any non-numeric data can be used to 'finish' the interaction. Test your algorithm with the graph from exercise 4a.

To test your program you can execute the dryrun program that corresponds to this exercise. It expects to find two programs in the current directory:

- `dijkstra.c` – your solution to this exercise
- `PQueue.c` – your solution to Exercise 4

You can use dryrun as follows:

```
prompt$ 9024 dryrun dijkstra
```

Note: Please ensure that your output follows exactly the format shown above.

[hide answer]

- a. Shortest path from vertex 0 to vertex 3 has total weight 5. There are really only two plausible choices: either the direct edge from vertex 0 to vertex 3, or two edges via vertex 2. It is easy to see that the total weight on the two-edge path (5) is less than the weight on the single edge (6).

Shortest path from vertex 0 to vertex 7 has total weight 13. Probably (although not necessarily) the best paths are also the most direct ones, which suggests 0-1-6-7, 0-1-5-7 and 0-2-4-7. If we sum the lengths on each of these paths, they are, respectively: 17, 13, 15. So we choose the shortest: 0-1-5-7.

- b. Initialisation:

```
vSet = { 0, 1, 2, 3, 4, 5, 6, 7 }
dist = [ 0, ∞, ∞, ∞, ∞, ∞, ∞, ∞ ]
pred = [ -1, -1, -1, -1, -1, -1, -1, -1 ]
```

The vertex in `vSet` with minimum `dist[]` is 0. Relaxation along the edges (0,1,5), (0,2,4) and (0,3,6) results in:

```
vSet = { 1, 2, 3, 4, 5, 6, 7 }
dist = [ 0, 5, 4, 6, ∞, ∞, ∞ ]
pred = [ -1, 0, 0, 0, -1, -1, -1 ]
```

Now the vertex in `vSet` with minimum `dist[]` is 2. Considering all edges from 2 to nodes still in `vSet`:

- relaxation along (2,1,8) does not give us a shorter distance to node 1
- relaxation along (2,3,1) yields a smaller value ($4+1=5$) for `dist[3]`, and `pred[3]` is updated to 2
- relaxation along (2,4,3) yields a smaller value ($4+3=7$) for `dist[4]`, and `pred[4]` is updated to 2
- relaxation along (2,5,7) yields a smaller value ($4+7=11$) for `dist[5]`, and `pred[5]` is updated to 2

```
vSet = { 1, 3, 4, 5, 6, 7 }
dist = [ 0, 5, 4, 5, 7, 11, ∞ ]
pred = [ -1, 0, 0, 2, 2, 2, -1 ]
```

Next, we could choose either 1 or 3, since both vertices have minimum distance 5. Suppose we choose 1. Relaxation along (1,5,2) and (1,6,7) results in new values for nodes 5 and 6:

```
vSet = { 3, 4, 5, 6, 7 }
dist = [ 0, 5, 4, 5, 7, 7, 12 ]
pred = [ -1, 0, 0, 2, 2, 1, -1 ]
```


Now we consider vertex 3. The only adjacent node still in vSet is 4, but there is no shorter path to 4 through 3. Hence no update to dist[] or pred[]:

```
vSet = { 4, 5, 6, 7 }
dist = [ 0, 5, 4, 5, 7, 7, 12, ∞ ]
pred = [ -1, 0, 0, 2, 2, 1, 1, -1 ]
```

Next we could choose either vertex 4 or 5. Suppose we choose 4. Edge (4,7,8) is the only one that leads to an update:

```
vSet = { 5, 6, 7 }
dist = [ 0, 5, 4, 5, 7, 7, 12, 15 ]
pred = [ -1, 0, 0, 2, 2, 1, 1, 4 ]
```

Vertex 5 is next. Relaxation along edges (5,6,3) and (5,7,6) results in:

```
vSet = { 6, 7 }
dist = [ 0, 5, 4, 5, 7, 7, 10, 13 ]
pred = [ -1, 0, 0, 2, 2, 1, 5, 5 ]
```

Of the two vertices left in vSet, 6 has the shorter distance. Edge (6,7,5) does not update the values for node 7 since $\text{dist}[7]=13 < \text{dist}[6]+5=15$. Hence:

```
vSet = { 7 }
dist = [ 0, 5, 4, 5, 7, 7, 10, 13 ]
pred = [ -1, 0, 0, 2, 2, 1, 5, 5 ]
```

Processing the last remaining vertex in vSet will obviously not change anything. The values in pred[] determine shortest paths to all nodes as follows:

```
0: distance = 0, shortest path: 0
1: distance = 5, shortest path: 0-1
2: distance = 4, shortest path: 0-2
3: distance = 5, shortest path: 0-2-3
4: distance = 7, shortest path: 0-2-4
5: distance = 7, shortest path: 0-1-5
6: distance = 10, shortest path: 0-1-5-6
7: distance = 13, shortest path: 0-1-5-7
```

c. Sample implementation of Dijkstra's algorithm, including a recursive function to display a path via pred[]:

```
void showPath(int v, int pred[]) {
    if (pred[v] == -1) {
        printf("%d", v);
    } else {
        showPath(pred[v], pred);
        printf("-%d", v);
    }
}

void dijkstraSSSP(Graph g, Vertex source) {
    int dist[MAX_NODES];
    int pred[MAX_NODES];
    bool vSet[MAX_NODES]; // vSet[v] = true <=> v has not been processed
    int s, t;

    PQQueueInit();
    int nV = numVertices(g);
    for (s = 0; s < nV; s++) {
        joinPQueue(s);
        dist[s] = VERY_HIGH_VALUE;
        pred[s] = -1;
        vSet[s] = true;
    }
    dist[source] = 0;
    while (!PQueueIsEmpty()) {
        s = leavePQueue(dist);
        vSet[s] = false;
        for (t = 0; t < nV; t++) {
            if (vSet[t]) {
                int weight = adjacent(g,s,t);
                if (weight > 0 && dist[s]+weight < dist[t]) { // relax along (s,t,weight)
                    dist[t] = dist[s] + weight;
                    pred[t] = s;
                }
            }
        }
    }
    for (s = 0; s < nV; s++) {
        printf("%d: ", s);
        if (dist[s] < VERY_HIGH_VALUE) {
            printf("distance = %d, shortest path: ", dist[s]);
        }
    }
}
```

```

        showPath(s, pred);
        putchar('\n');
    } else {
        printf("no path\n");
    }
}
}

```

6. (All-pair shortest paths)

- In the graph from exercise 5a, what is the shortest path from vertex 3 to vertex 6? From vertex 6 to vertex 3? How did you determine these?
- Trace the execution of Floyd's algorithm on the graph from exercise 5a to compute the shortest paths between *all* pairs of vertices.

[\[hide answer\]](#)

- Shortest path from vertex 3 to vertex 6 has total weight 10. The most direct paths use three edges, e.g. 3-0-1-6, 3-2-1-6 or 3-2-5-6. If we sum the lengths on each of these paths, the shortest one is 3-2-5-6 with total weight 11. However, there is a path from vertex 2 to vertex 5 that is shorter than the direct edge, via vertex 4. This reduces the total weight of the path from 3 to 6 (now via 2, 4 and 5) to $1+3+3+3=10$.

Since all weighted edges in the graph are bidirectional, the shortest path from vertex 6 to vertex 3 is the reverse of the shortest path we found from 3 to 6: 6-5-4-2-3. Its total weight is the same (10).

b. Initialisation:

dist	[0]	[1]	[2]	[3]	[4]	[5]	[6]	[7]	path	[0]	[1]	[2]	[3]	[4]	[5]	[6]	[7]
[0]	0	5	4	6	∞	∞	∞	∞	[0]	-	1	2	3	-	-	-	-
[1]	5	0	8	∞	∞	2	7	∞	[1]	0	-	2	-	-	5	6	-
[2]	4	8	0	1	3	7	∞	∞	[2]	0	1	-	3	4	5	-	-
[3]	6	∞	1	0	6	∞	∞	∞	[3]	0	-	2	-	4	-	-	-
[4]	∞	∞	3	6	0	3	∞	8	[4]	-	-	2	3	-	5	-	7
[5]	∞	2	7	∞	3	0	3	6	[5]	-	1	2	-	4	-	6	7
[6]	∞	7	∞	∞	∞	3	0	5	[6]	-	1	-	-	-	5	-	7
[7]	∞	∞	∞	∞	8	6	5	0	[7]	-	-	-	-	4	5	6	-

After 1st iteration $i=0$:

dist	[0]	[1]	[2]	[3]	[4]	[5]	[6]	[7]	path	[0]	[1]	[2]	[3]	[4]	[5]	[6]	[7]
[0]	0	5	4	6	∞	∞	∞	∞	[0]	-	1	2	3	-	-	-	-
[1]	5	0	8	11	∞	2	7	∞	[1]	0	-	2	0	-	5	6	-
[2]	4	8	0	1	3	7	∞	∞	[2]	0	1	-	3	4	5	-	-
[3]	6	∞	1	0	6	∞	∞	∞	[3]	0	-	2	-	4	-	-	-
[4]	∞	∞	3	6	0	3	∞	8	[4]	-	-	2	3	-	5	-	7
[5]	∞	2	7	∞	3	0	3	6	[5]	-	1	2	-	4	-	6	7
[6]	∞	7	∞	∞	∞	3	0	5	[6]	-	1	-	-	-	5	-	7
[7]	∞	∞	∞	∞	8	6	5	0	[7]	-	-	-	-	4	5	6	-

After 2nd iteration $i=1$:

dist	[0]	[1]	[2]	[3]	[4]	[5]	[6]	[7]	path	[0]	[1]	[2]	[3]	[4]	[5]	[6]	[7]
[0]	0	5	4	6	∞	7	12	∞	[0]	-	1	2	3	-	1	1	-
[1]	5	0	8	11	∞	2	7	∞	[1]	0	-	2	0	-	5	6	-
[2]	4	8	0	1	3	7	15	∞	[2]	0	1	-	3	4	5	1	-
[3]	6	11	1	0	6	13	18	∞	[3]	0	0	2	-	4	0	0	-
[4]	∞	∞	3	6	0	3	∞	8	[4]	-	-	2	3	-	5	-	7
[5]	7	2	7	13	3	0	3	6	[5]	1	1	2	1	4	-	6	7
[6]	12	7	15	18	∞	3	0	5	[6]	1	1	1	1	-	5	-	7

[7]	∞	∞	∞	∞	8	6	5	0	[7]	–	–	–	–	4	5	6	–
-----	----------	----------	----------	----------	---	---	---	---	-----	---	---	---	---	---	---	---	---

After 3rd iteration i=2:

dist	[0]	[1]	[2]	[3]	[4]	[5]	[6]	[7]	path	[0]	[1]	[2]	[3]	[4]	[5]	[6]	[7]
[0]	0	5	4	5	7	7	12	∞	[0]	–	1	2	2	2	1	1	–
[1]	5	0	8	9	11	2	7	∞	[1]	0	–	2	2	2	5	6	–
[2]	4	8	0	1	3	7	15	∞	[2]	0	1	–	3	4	5	1	–
[3]	5	9	1	0	4	8	16	∞	[3]	2	2	2	–	2	2	2	–
[4]	7	11	3	4	0	3	18	8	[4]	2	2	2	2	–	5	2	7
[5]	7	2	7	8	3	0	3	6	[5]	1	1	2	2	4	–	6	7
[6]	12	7	15	16	18	3	0	5	[6]	1	1	1	1	1	5	–	7
[7]	∞	∞	∞	∞	8	6	5	0	[7]	–	–	–	–	4	5	6	–

After 4th iteration i=3: unchanged

After 5th iteration i=4:

dist	[0]	[1]	[2]	[3]	[4]	[5]	[6]	[7]	path	[0]	[1]	[2]	[3]	[4]	[5]	[6]	[7]
[0]	0	5	4	5	7	7	12	15	[0]	–	1	2	2	2	1	1	2
[1]	5	0	8	9	11	2	7	19	[1]	0	–	2	2	2	5	6	2
[2]	4	8	0	1	3	6	15	11	[2]	0	1	–	3	4	4	1	4
[3]	5	9	1	0	4	7	16	12	[3]	2	2	2	–	2	2	2	2
[4]	7	11	3	4	0	3	18	8	[4]	2	2	2	2	–	5	2	7
[5]	7	2	6	7	3	0	3	6	[5]	1	1	4	4	4	–	6	7
[6]	12	7	15	16	18	3	0	5	[6]	1	1	1	1	1	5	–	7
[7]	15	19	11	12	8	6	5	0	[7]	4	4	4	4	4	5	6	–

After 6th iteration i=5:

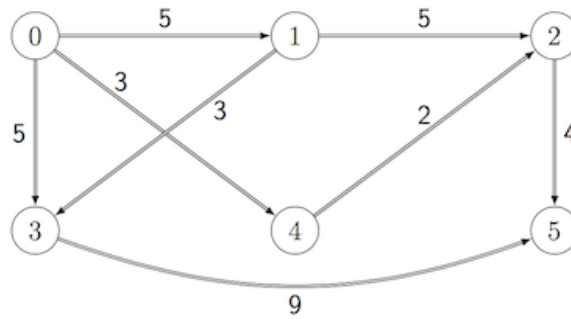
dist	[0]	[1]	[2]	[3]	[4]	[5]	[6]	[7]	path	[0]	[1]	[2]	[3]	[4]	[5]	[6]	[7]
[0]	0	5	4	5	7	7	10	13	[0]	–	1	2	2	2	1	1	1
[1]	5	0	8	9	5	2	5	8	[1]	0	–	2	2	5	5	5	5
[2]	4	8	0	1	3	6	9	11	[2]	0	1	–	3	4	4	4	4
[3]	5	9	1	0	4	7	10	12	[3]	2	2	2	–	2	2	2	2
[4]	7	5	3	4	0	3	6	8	[4]	2	5	2	2	–	5	5	7
[5]	7	2	6	7	3	0	3	6	[5]	1	1	4	4	4	–	6	7
[6]	10	5	9	10	6	3	0	5	[6]	5	5	5	5	5	5	–	7
[7]	13	8	11	12	8	6	5	0	[7]	5	5	4	4	4	5	6	–

The last two iterations (i=6, 7) do not effect any more changes.

Observe that the final distances and shortest paths between 0 and all other vertices are the same as in the solution to exercise 5a, as it should be.

7. (Maximum flow)

- a. Identify a maximum flow from source=0 to sink=5 in the following network (without applying the algorithm from the lecture):



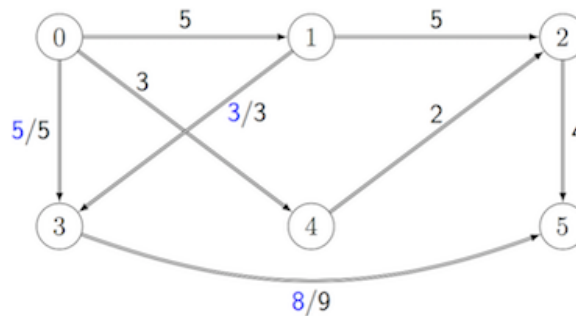
What approach did you use in finding the maxflow?

- b. Show how Edmonds and Karp's algorithm would construct a maximum flow from 0 to 5 for the above network. How many augmenting paths do you have to consider?

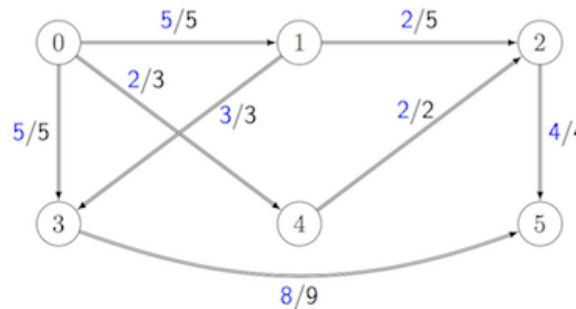
[hide answer]

- a. I suspect (no proof) that most people would use a strategy like:

1. Choose an edge from a vertex into the sink and attempt to see if and how the maximum flow through this edge can be achieved, for example edge (3,5) with capacity 9.
2. Attempt to maximise the flow into this vertex 3 to check if the required capacity 9 can be reached.
3. Realise that the maximum flow into 3 is actually 8, which would get you to a partial solution like:



4. Inspect the other edge into the sink – (2,5) with capacity 4 – and attempt to max out its capacity.
5. Realise that the maximum flow that can be sent into node 2 is indeed 4, which results in the maximum flow shown below.



- b. The shortest augmenting path found by BFS is 0-3-5, along which we can send $df=5$. The resulting `flow[ ][ ]` matrix is:

	[0]	[1]	[2]	[3]	[4]	[5]
[0]	0	0	0	5	0	0
[1]	0	0	0	0	0	0
[2]	0	0	0	0	0	0
[3]	-5	0	0	0	0	5
[4]	0	0	0	0	0	0
[5]	0	0	0	-5	0	0

The negative values encode the "reversed" edges in the residual graph.

If we choose neighbours in ascending order, then the next augmenting path found by BFS is 0-1-2-5 with $df=4$. We update the `flow[ ][ ]` matrix to:

	[0]	[1]	[2]	[3]	[4]	[5]
--	-----	-----	-----	-----	-----	-----

[0]	0	4	0	5	0	0
[1]	-4	0	4	0	0	0
[2]	0	-4	0	0	0	4
[3]	-5	0	0	0	0	5
[4]	0	0	0	0	0	0
[5]	0	0	-4	-5	0	0

Now the shortest augmenting path is 0-1-3-5 with $df=1$, so we update the `flow[ ][ ]` matrix as follows:

	[0]	[1]	[2]	[3]	[4]	[5]
[0]	0	5	0	5	0	0
[1]	-5	0	4	1	0	0
[2]	0	-4	0	0	0	4
[3]	-5	-1	0	0	0	6
[4]	0	0	0	0	0	0
[5]	0	0	-4	-6	0	0

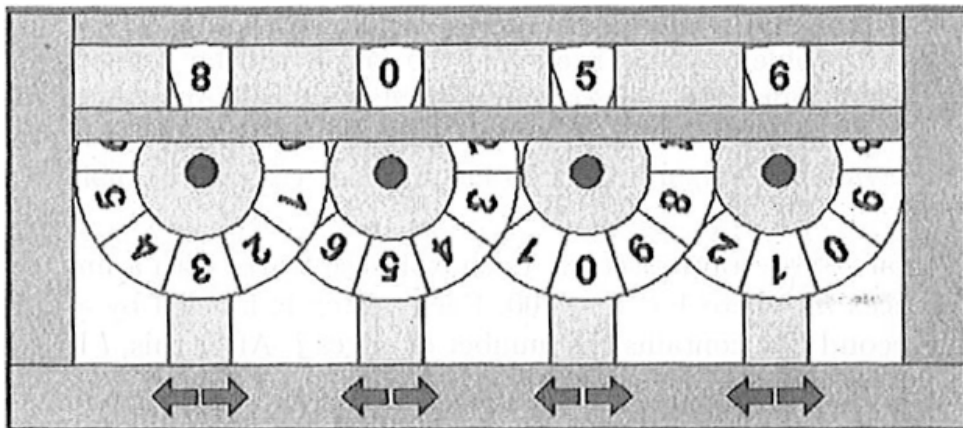
The residual graph now admits one more augmenting path: 0-4-2-1-3-5 with $df=2$, by which some of the current flow from 1 to 2 is "redirected" to 3 and replaced by the same amount from 4 to 2. This results in the `flow[ ][ ]` matrix:

	[0]	[1]	[2]	[3]	[4]	[5]
[0]	0	5	0	5	2	0
[1]	-5	0	2	3	0	0
[2]	0	-2	0	0	-2	4
[3]	-5	-3	0	0	0	8
[4]	-2	0	2	0	0	0
[5]	0	0	-4	-8	0	0

No more augmenting path exists. The maximum flow is $5+4+1+2 = 12$, the sum of what we could maximally send along the 4 augmenting paths that we computed.

8. Challenge Exercise

Consider a machine with four wheels. Each wheel has the digits 0...9 printed clockwise on it. The current *state* of the machine is given by the four topmost digits *abcd*, e.g. 8056 in the picture below.



Each wheel can be controlled by two buttons: Pressing the button labelled with " $\leftarrow$ " turns the corresponding wheel clockwise one digit ahead, whereas pressing " $\rightarrow$ " turns it anticlockwise one digit back.

Write a C-program to determine the *minimum* number of button presses required to transform

- a given initial state, *abcd*
- to a given goal state, *efgh*
- without passing through any of $n \geq 0$ given "forbidden" states, $\text{forbidden}[] = \{w_1x_1y_1z_1, \dots, w_nx_ny_nz_n\}$.

For example, the state 8056 as depicted can be transformed into 0056 in 2 steps if `forbidden[] = {}`, whereas a minimum of 4 steps is needed for the same task if `forbidden[] = {9056}`. (Why?)

Use your program to compute the least number of button presses required to transform 8056 to 7012 if

- there are no forbidden states
- you are not permitted to pass through any state 7055–8055 (i.e., 7055, 7056, ..., 8055 all are forbidden)
- you are not permitted to pass through any state 0000–0999 or 7055–8055

[\[hide answer\]](#)

The problem can be solved by a breadth-first search on an undirected graph:

- nodes 0...9999 correspond to the possible configurations 0000 – 9999 of the machine;
- edges connect nodes v and w if, and only if, one button press takes the machine from v to w and vice versa.

The following code implements BFS on this graph with the help of the integer queue ADT from the lecture ([queue.h](#), [queue.c](#)). Note that the graph is built *implicitly*: nodes are generated as they are encountered during the search.

```
#include "queue.h"

int shortestPath(int source, int target, int forbidden[], int n) {
    int visited[10000];

    int i;
    for (i = 0; i <= 9999; i++)
        visited[i] = -1; // mark all nodes as unvisited
    for (i = 0; i < n; i++)
        visited[forbidden[i]] = -2; // mark forbidden nodes as visited => they won't be selected
    visited[source] = source;

    queue Q = newQueue();
    QueueEnqueue(Q, source);
    bool found = (target == source);
    while (!found && !QueueIsEmpty(Q)) {
        int v = QueueDequeue(Q);
        if (v == target) {
            found = true;
        } else {
            int wheel, turn;
            for (wheel = 10; wheel <= 10000; wheel *= 10) { // fancy way of generating the
                for (turn = 1; turn <= 9; turn += 8) { // eight neighbour configurations of v
                    int w = wheel * (v / wheel) + (v % wheel + (wheel/10) * turn) % wheel;
                    if (visited[w] == -1) {
                        visited[w] = v;
                        QueueEnqueue(Q, w);
                    }
                }
            }
        }
    }
    dropQueue(Q);
    if (found) { // unwind path to determine length
        int length = 0;
        while (target != source) {
            target = visited[target]; // move to predecessor on path
            length++;
        }
        return length;
    } else {
        return -1; // no solution
    }
}
```

- 9
- 17
- ∞ (unreachable)

Assessment

Group submissions will not be allowed. Your program must be entirely your own work. Plagiarism detection software will be used to compare all submissions pairwise.

Submit your solutions to **Exercise 4 and 5** using the following **give** command:

```
prompt$ give cs9024 week6 PQueue.c dijkstra.c
```

Make sure you spell the filenames correctly. You can run **give** multiple times. Only your last submission will be marked.

The deadline for submission is **Wednesday, 30 October 11:00:00am**.

Each program is worth 1 mark. Total marks = 2. Auto-marking will be run by the lecturer several days after the submission deadline using different test cases than `dryrun` does.

Hints:

- Programs will not be manually marked.
- It is important that the output of your program follows exactly the format shown above, otherwise auto-marking will result in 0 marks.
- Ensure that your program compiles on a CSE machine with the standard options `-Wall -Werror -std=c11`. Programs that do not compile will receive 0 marks.
- `dryrun` and auto-marking also check the correct usage of dynamic memory allocation and pointers as well as the absence of memory leaks.
- Do your own testing in addition to running `dryrun`.